

TP -- OSINT

Renseignement en sources ouvertes

Appliquer requests, BeautifulSoup4 et Scrapy au renseignement sur des cibles publiques : domaines, entites, veille presse.

5 heures de developpement

Qu'est-ce que l'OSINT ?

L'OSINT (Open Source Intelligence) est la collecte et l'analyse systematique d'informations provenant de sources publiquement accessibles. Utilisee en cybersécurité défensive, compliance, journalisme d'investigation et due diligence financière.

Dans ce TP, vous re-utilisez exactement les outils des trois premiers jours (requests+BS4, Scrapy) dans un contexte professionnel concret.

CADRE LEGAL -- A RESPECTER ABSOLUMENT

1. Sources UNIQUEMENT publiques -- aucune authentification, aucun bypass
 2. Données editoriales ou registres officiels -- pas de données personnelles nominatives
 3. RGPD Art. 6 : base légale = intérêt légitime (veille concurrentielle documentée)
 4. robots.txt respecte pour chaque cible -- User-Agent identifiable obligatoire
 5. Throttling >= 1s entre requêtes -- ne jamais dégrader le service cible
- VIOLATION = code pénal art. 323-1 (STAD) si accès non autorisé

LIVRABLES ATTENDUS

td41_domaine.py -> rapport_domaine.json (empreinte technique)
td42_entite.py -> fiche_entite.json (cartographie publique)
veille/ (Scrapy) -> mentions.csv + veille.db (surveillance RSS)
ETHIQUE.md : 3 réponses (droit, personnel, discret) pour chaque TD

TD 4.1 -- Empreinte d'un domaine

1 h 30

Contexte : un client vous demande d'évaluer l'exposition technique d'un concurrent avant une acquisition. Vous analysez son domaine via des sources publiques : WHOIS, headers HTTP, journaux de certificats (Certificate Transparency).

Etape 1 -- WHOIS et headers HTTP

```
pip install python-whois requests

# td41_domaine.py
import requests, json, socket
import whois # pip install python-whois

HEADERS = {"User-Agent": "IPSSI-OSINT (+cours@ipssi.fr)"}

def analyse_whois(domaine: str) -> dict:
    try:
        w = whois.whois(domaine)
        return {
            "registrar"      : str(w.registrar or "n/a"),
            "creation_date"  : str(w.creation_date or "n/a")[:10],
            "expiration_date": str(w.expiration_date or "n/a")[:10],
            "name_servers"   : list(set(w.name_servers or [])),
            "country"        : str(w.country or "n/a"),
        }
    except Exception as e:
        return {"erreur": str(e)}

def analyse_headers(domaine: str) -> dict:
    try:
        r = requests.head(f"https://{domaine}", headers=HEADERS, timeout=10,
                           allow_redirects=True)
        h = r.headers
        return {
            "status"        : r.status_code,
            "server"        : h.get("Server", "n/a"),
            "x_powered_by"  : h.get("X-Powered-By", "n/a"),
            "x_frame_options": h.get("X-Frame-Options", "n/a"),
            "csp_present"   : "Content-Security-Policy" in h,
            "hsts_present"  : "Strict-Transport-Security" in h,
        }
    except Exception as e:
        return {"erreur": str(e)}
```

Etape 2 -- Sous-domaines via Certificate Transparency

crt.sh est une base de données publique des certificats TLS émis. Elle expose une API JSON permettant de lister tous les sous-domaines pour lesquels un certificat a été demandé.

```

import time

def sous_domaines_crtsh(domaine: str) -> list[str]:
    """Cherche les sous-domaines via l'API publique crt.sh"""
    url = f"https://crt.sh/?q=%.{domaine}&output=json"
    try:
        r = requests.get(url, headers=HEADERS, timeout=20)
        r.raise_for_status()
        data = r.json()
        # Filtrer les wildcards et dedoubler
        subs = list(set(
            entry["name_value"]
            for entry in data
            if "*" not in entry["name_value"]
            and entry["name_value"].endswith(domaine)
        ))
        return sorted(subs)[:100] # limiter a 100
    except Exception as e:
        return [f"ERREUR: {e}"]

```

Etape 3 -- robots.txt + IP + rapport complet

```

def analyse_robots(domaine: str) -> str:
    try:
        r = requests.get(f"https://{domaine}/robots.txt",
                        headers=HEADERS, timeout=10)
        return r.text[:1000] if r.status_code == 200 else f"HTTP {r.status_code}"
    except Exception as e:
        return str(e)

def analyser_domaine(domaine: str) -> dict:
    print(f"[*] Analyse de {domaine}...")
    rapport = {
        "domaine"      : domaine,
        "ip"           : socket.gethostbyname(domaine) if domaine else "n/a",
        "whois"        : analyse_whois(domaine),
        "headers_http" : analyse_headers(domaine),
        "sous_domaines": sous_domaines_crtsh(domaine),
        "robots_txt"   : analyse_robots(domaine),
    }
    rapport["nb_sous_domaines"] = len(rapport["sous_domaines"])
    return rapport

if __name__ == "__main__":
    import sys
    domaine = sys.argv[1] if len(sys.argv) > 1 else "wikipedia.org"
    time.sleep(1) # politesse
    rapport = analyser_domaine(domaine)
    sortie = f"rapport_{domaine}.json"
    with open(sortie, "w", encoding="utf-8") as f:
        json.dump(rapport, f, indent=2, ensure_ascii=False)
    print(f"[+] Rapport sauvegarde : {sortie}")
    print(f"    {rapport['nb_sous_domaines']} sous-domaines trouvees")
    print(f"    Serveur : {rapport['headers_http'].get('server', 'n/a')}")

```

Questions d'analyse a inclure dans ETHIQUE.md

1. Ai-je le droit ? crt.sh et WHOIS sont des registres publics -- oui
 2. Est-ce personnel ? Non : sous-domaines et infos techniques
 3. Suis-je discret ? User-Agent identifiable + sleep entre requetes
- BONUS : que revele la liste des sous-domaines sur l'architecture interne ?

TD 4.2 -- Cartographie d'une entite publique

1 h 30

Choisissez une entreprise cotee au CAC40 ou une institution publique (ex: TotalEnergies, Airbus, BNP Paribas, Ministere de l'Economie). Construisez une fiche de renseignement a partir de trois sources publiques.

Source 1 -- Annuaire Entreprises data.gouv.fr

```

pip install requests feedparser

# td42_entite.py
import requests, json, time
from bs4 import BeautifulSoup

HEADERS = {"User-Agent": "IPSSI-OSINT (+cours@ipssi.fr)"}

def chercher_sirene(nom: str) -> dict:
    """API SIRENE officielle -- pas besoin de cle API"""
    url = f"https://api.annuaire-entreprises.data.gouv.fr/v3/search?q={nom}&limit=1"
    try:
        r = requests.get(url, headers=HEADERS, timeout=10)
        r.raise_for_status()
        data = r.json()
        if data.get("results"):
            ent = data["results"][0]
            return {
                "siren"           : ent.get("siren"),
                "denomination"    : ent.get("nom_complet"),
                "adresse_siege"   : ent.get("siege", {}).get("adresse"),
                "code_naf"        : ent.get("activite_principale"),
                "date_creation"    : ent.get("date_creation"),
                "tranche_effectif": ent.get("tranche_effectif_salarie"),
            }
        return {"resultat": "Non trouve dans SIRENE"}
    except Exception as e:
        return {"erreur": str(e)}

```

Source 2 -- Wikipedia (infobox + intro)

```
def scraper_wikipedia(nom: str) -> dict:
    """Scraper l'infobox et l'intro de la page Wikipedia"""
    slug = nom.replace(" ", "_")
    url = f"https://fr.wikipedia.org/wiki/{slug}"
    try:
        r = requests.get(url, headers=HEADERS, timeout=10)
        soup = BeautifulSoup(r.text, "lxml")

        # Infobox : tableau d'informations sur la droite
        infobox = {}
        table = soup.select_one("table.infobox, table.wikitable")
        if table:
            for tr in table.select("tr"):
                th = tr.select_one("th")
                td = tr.select_one("td")
                if th and td:
                    cle = th.get_text(strip=True)
                    val = td.get_text(" ", strip=True)[:200]
                    infobox[cle] = val

        # Premier paragraphe = introduction
        intro = ""
        for p in soup.select("#mw-content-text p"):
            txt = p.get_text(strip=True)
            if len(txt) > 80:
                intro = txt[:500]; break

        return {"infobox": infobox, "intro": intro, "url": url}
    except Exception as e:
        return {"erreur": str(e)}
```

Source 3 -- Flux RSS presse (Google News)

```
import feedparser

def veille_presse(nom: str, nb_max: int = 10) -> list[dict]:
    """Google News RSS : articles recents mentionnant l'entite"""
    query = nom.replace(" ", "+")
    url = f"https://news.google.com/rss/search?q={query}&hl=fr&gl=FR&ceid=FR:fr"
    feed = feedparser.parse(url)
    return [
        {
            "titre" : e.get("title", ""),
            "source": e.get("source", {}).get("title", ""),
            "date"   : e.get("published", ""),
            "lien"   : e.get("link", ""),
        }
        for e in feed.entries[:nb_max]
    ]
```

Agregation en fiche complete

```
def construire_fiche(nom: str) -> dict:
    print(f"[*] Construction de la fiche pour : {nom}")
    fiche = {"entite": nom}
    fiche["sirene"] = chercher_sirene(nom); time.sleep(1)
    fiche["wikipedia"] = scraper_wikipedia(nom); time.sleep(1)
    fiche["presse"] = veille_presse(nom)
    fiche["nb_articles"] = len(fiche["presse"])
    return fiche

if __name__ == "__main__":
    import sys
    nom = " ".join(sys.argv[1:]) if len(sys.argv) > 1 else "TotalEnergies"
    fiche = construire_fiche(nom)
    with open("fiche_entite.json", "w", encoding="utf-8") as f:
        json.dump(fiche, f, indent=2, ensure_ascii=False)
    print(f"[+] Fiche sauvegardee : fiche_entite.json")
    print(f"    SIREN    : {fiche['sirene'].get('siren', 'n/a')}")
    print(f"    Articles: {fiche['nb_articles']}")
```

TD 4.3 -- Veille automatisee avec Scrapy

2 heures

Construisez un spider Scrapy qui surveille 5 flux RSS de medias francophones et detecte les mentions d'une cible. Un score d'alerte simple (positif/negatif) est calcule sur le texte de chaque article.

Setup du projet

```
scrapy startproject veille
cd veille
scrapy genspider rss_spider rss
```

items.py

```
import scrapy

class MentionItem(scrapy.Item):
    titre = scrapy.Field()
    url = scrapy.Field()
    source = scrapy.Field()
    date_publi = scrapy.Field()
    resume = scrapy.Field()
    score_alerte = scrapy.Field() # 0=neutre 1=negatif 2=positif
```

Spider RSS multi-sources

```

# veille/spiders/rss_spider.py
import scrapy
from veille.items import MentionItem

CIBLE = "TotalEnergies" # mot-cle a surveiller (modifiable)

MOTS_NEGATIFS = ["fraude", "amende", "condamne", "scandale", "plainte",
                 "liquidation", "faillite", "perquisition", "accuse"]
MOTS_POSITIFS = ["croissance", "benefice", "record", "acquisition", "innovation",
                 "nomination", "partenariat", "expansion", "investissement"]

FLUX_RSS = [
    "https://www.lemonde.fr/rss/une.xml",
    "https://www.lesechos.fr/rss/rss_une.xml",
    "https://www.lefigaro.fr/rss/figaro_actualites.xml",
    "https://www.bfmtv.com/rss/info/flux-rss/flux-toutes-les-actualites/",
    "https://www.01net.com/feed/"
]

class RssSpider(scrapy.Spider):
    name = "rss_spider"
    start_urls = FLUX_RSS
    custom_settings = {
        "ROBOTSTXT_OBEY" : True,
        "DOWNLOAD_DELAY" : 1.0,
        "RANDOMIZE_DOWNLOAD_DELAY": True,
        "USER_AGENT"      : "IPSSI-OSINT-veille (+cours@ipssi.fr)",
        "LOG_LEVEL"       : "INFO",
    }

    def parse(self, response):
        # Parser flux RSS/Atom
        for item in response.xpath("//item | //entry"):
            titre = item.xpath("title/text()").get("").strip()
            resume = item.xpath("description/text() | summary/text()").get("").strip()[:300]

            # Filtrer uniquement les articles mentionnant la cible
            if CIBLE.lower() not in (titre + resume).lower():
                continue

            url = item.xpath("link/text() | link/@href").get("").strip()
            date_pub = item.xpath("pubDate/text() | published/text()").get("").strip()
            source = response.url.split("/")[2]

            # Calcul du score d'alerte
            texte = (titre + " " + resume).lower()
            neg = sum(1 for m in MOTS_NEGATIFS if m in texte)
            pos = sum(1 for m in MOTS_POSITIFS if m in texte)
            score = 1 if neg > pos else (2 if pos > neg else 0)

            yield MentionItem(
                titre=titre, url=url, source=source,
                date_publi=date_pub, resume=resume, score_alerte=score,
            )

```

pipelines.py -- CleanPipeline + SQLitePipeline

```
# veille/pipelines.py
import sqlite3
from itemadapter import ItemAdapter

DDL = '''CREATE TABLE IF NOT EXISTS mentions (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    titre         TEXT NOT NULL,
    url           TEXT UNIQUE,
    source        TEXT,
    date_publi    TEXT,
    resume        TEXT,
    score_alerte  INTEGER DEFAULT 0,
    scraped_at    TEXT DEFAULT CURRENT_TIMESTAMP
)'''

class CleanPipeline:
    def process_item(self, item, spider):
        a = ItemAdapter(item)
        a["titre"] = a.get("titre", "").strip()
        a["resume"] = a.get("resume", "").strip()[:300]
        return item

class SQLitePipeline:
    def open_spider(self, spider):
        self.cx = sqlite3.connect("veille.db")
        self.cx.execute(DDL); self.cx.commit()

    def process_item(self, item, spider):
        a = ItemAdapter(item)
        try:
            self.cx.execute(
                "INSERT OR IGNORE INTO mentions "
                "(titre,url,source,date_publi,resume,score_alerte) "
                "VALUES(?,?,?,?,?,?)",
                (a["titre"], a.get("url", ""), a.get("source", ""),
                 a.get("date_publi", ""), a.get("resume", ""), a.get("score_alerte", 0)))
            self.cx.commit()
        except sqlite3.Error as e:
            spider.logger.error(f"SQLite: {e}")
            return item

    def close_spider(self, spider):
        n = self.cx.execute("SELECT COUNT(*) FROM mentions").fetchone()[0]
        spider.logger.info(f"[OSINT] {n} mentions en base")
        self.cx.close()
```

settings.py + lancement

```
# veille/settings.py
ITEM_PIPELINES = {
    "veille.pipelines.CleanPipeline" : 100,
    "veille.pipelines.SQLitePipeline": 200,
}
FEEDS = {
    "mentions.csv": {"format": "csv", "encoding": "utf-8", "overwrite": True},
}
```

```
# Lancer la veille
scrapy crawl rss_spider -L INFO

# Analyser les resultats
python3 -c "
import sqlite3
cx = sqlite3.connect('veille.db')
rows = cx.execute(
    'SELECT titre, source, score_alerte FROM mentions ORDER BY score_alerte DESC'
).fetchall()
print(f'{len(rows)} mentions trouvees')
for r in rows[:5]:
    print(f' [{r[2]]} {r[0][:60]} ({r[1]})')
cx.close()
"
```

CHECKLIST AVANT RENDU

- [] rapport_domaine.json : WHOIS + sous-domaines crt.sh + headers HTTP
- [] fiche_entite.json : SIREN + Wikipedia infobox + articles presse
- [] veille.db : table mentions avec UNIQUE(url) et score_alerte
- [] mentions.csv exportee par Scrapy
- [] ETHIQUE.md : 3 reponses (droit, personnel, discret) pour CHAQUE TD
- [] User-Agent identifiable dans tous les scripts
- [] sleep >= 1s entre requetes dans td41 et td42

Pour aller plus loin -- Defis autonomes

Ces defis poussent l'OSINT vers l'analyse critique et l'interpretation contextuelle -- des taches qui requierent votre jugement humain sur des donnees reelles que vous avez vous-meme collectees.

Defi 1 -- Calibrer votre scoring de sentiment

Votre spider Scrapy attribue un score d'alerte (0/1/2) base sur des listes de mots-cles. Ce systeme est naif. Evaluez sa precision sur vos donnees reelles.

- Ouvrez 5 articles classes score=2 (positif) depuis mentions.csv. Lisez-les vraiment sur le site source. Combien sont reellement positifs pour votre cible ? Notez les faux positifs.
- Faites la meme chose pour 5 articles score=1 (negatif). Quel est votre taux de precision reel (vrais positifs / total) ?
- Ajoutez 3 mots a MOTS_NEGATIFS et 3 mots a MOTS_POSITIFS issus de vos observations. Relancez la veille et mesurez si la precision s'ameliore.

Pourquoi ce defi resiste a l'IA

L'IA ne peut pas lire les articles source en temps reel sur les sites.

La calibration depend de VOTRE cible specifique et du contexte sectoriel.

Le jugement sur le ton d'un article necessite de le lire -- pas de le deviner.

Defi 2 -- OSINT sur un domaine que vous connaissez

Choisissez un domaine que vous connaissez personnellement : votre ecole, une entreprise de stage, un client potentiel, votre propre site si vous en avez un. Lancez `td41_domaine.py` sur ce domaine.

- Quelles informations vous surprennent dans le rapport ? Combien de sous-domaines que vous ne connaissiez pas ont été trouvés ?
- Le serveur web est-il identifié par les headers HTTP ? Cette information est-elle utile pour un attaquant ?
- Y a-t-il des sous-domaines de preprod ou staging exposés ? Si oui, cela constitue un risque -- lequel exactement ?

Rédigez un paragraphe de 5 lignes 'Ce qu'un auditeur externe apprendrait en 5 minutes sur ce domaine' et partagez-le avec votre professeur.

Defi 3 -- Croiser veille et historique Wikipedia

Choisissez un article de votre `veille.db` avec un `score_alerte` élevé (1 ou 2). Retrouvez l'événement réel qui a généré cet article, puis vérifiez s'il a été enregistré dans l'historique Wikipedia de votre entité.

- Ouvrez la page Wikipedia de votre entité, onglet 'Voir l'historique'. Cherchez une modification faite autour de la date de l'article de presse.
- Lisez le diff de cette modification Wikipedia (lien 'cur' ou 'prev'). Quelle information a été ajoutée ou corrigée ?
- Conclusion : Wikipedia est-il une source OSINT fiable pour la veille temps-réel ? Pourquoi ou pourquoi pas ?